

High-Level Design

Zero-Trust Federated Aggregation for Health IoT Metrics

via Smart Contracts & Fully Homomorphic Encryption

Document	High-Level Design (HLD)
Version	0.1 — Draft for technical review
Date	May 03, 2026
Status	For Review
Audience	Engineering, Cryptography, Security review
Cipher	CKKS (RNS variant) over $\mathbb{Z}_q[X]/(X^{N+1})$, $N = 2^{15}$
ZK system	Groth16 over BN254 (one-time per-circuit setup)
Settlement	EVM L2 (Polygon zkEVM recommended)

This document specifies the system architecture, cryptographic protocol, smart-contract design, and threat model for a privacy-preserving federated learning platform on health IoT data. The CKKS engine is exclusively client-side; the aggregator never holds any secret key; ciphertexts never go on-chain.

1. Executive Summary

This system enables multiple healthcare-IoT clients to collaboratively train a shared machine-learning model over their private patient streams without ever exposing raw data, raw weights, or raw gradients to any party — including each other and the aggregating server. It combines four primitives:

- **Local training** on each client's own labeled IoT stream (PyTorch, on-device).
- **Fully Homomorphic Encryption (CKKS)** applied *only* on the client. Each client encrypts its local model weights before transmission. The aggregator operates exclusively on ciphertexts and never holds a secret key.
- **Homomorphic FedAvg** on the server: the aggregator computes $c_agg = (1/N) \cdot \sum_i c_i$ entirely in the encrypted domain, requiring only ciphertext-addition and a single plaintext-scalar multiplication (multiplicative depth = 1; no bootstrapping needed).
- **zk-SNARK-verified aggregation**: after off-chain aggregation, the server produces a Groth16 proof attesting that c_agg is the correct homomorphic mean of the clients' commitments. A smart contract verifies the proof in ~250k gas. Ciphertexts never go on-chain — only Poseidon hashes and the constant-size proof.

Inference is purely local: each client runs the latest decrypted global model on its incoming unlabeled IoT stream. No inference traffic crosses the network.

What the design guarantees. An honest-but-curious or actively malicious aggregator cannot (a) read any client's weights, (b) silently substitute or drop client contributions, or (c) misreport the aggregation result. Any of these violations are detected when the smart contract verifies the SNARK against the client commitments published on-chain.

What the design does *not* guarantee in v1. Defence against malicious clients submitting deliberately poisoned updates is deferred to v2 (client-side ZK range-proofs and Krum / Trimmed-Mean robust aggregation). Differential privacy noise injection is orthogonal and trivially layerable.

2. Goals and Non-Goals

2.1 Goals

- End-to-end privacy of patient-level data and per-client weight updates.
- Zero-trust posture: no single party — server, chain operator, peer client — sees another party's plaintext weights or data.
- Cryptographic verifiability of the aggregation step (not merely audit logs).
- Constant on-chain cost regardless of model dimension or number of clients.
- Support for both labeled local training and unlabeled local inference on a continuous IoT stream.
- Single-login portal experience for the client.

2.2 Non-Goals (v1)

- Robustness against malicious-client poisoning attacks.
- Differential-privacy guarantees (orthogonal — can layer in later).
- Decentralized aggregator (single coordinator with the SNARK as accountability layer).
- Multi-tenant isolation across organizations (single federation per deployment).
- Heterogeneous device fleet — client nodes are assumed to have non-trivial CPU/GPU (modern laptop or edge gateway). Watch / sensor itself is purely a producer.

3. System Overview

Three trust zones, three machines. Diagram below.

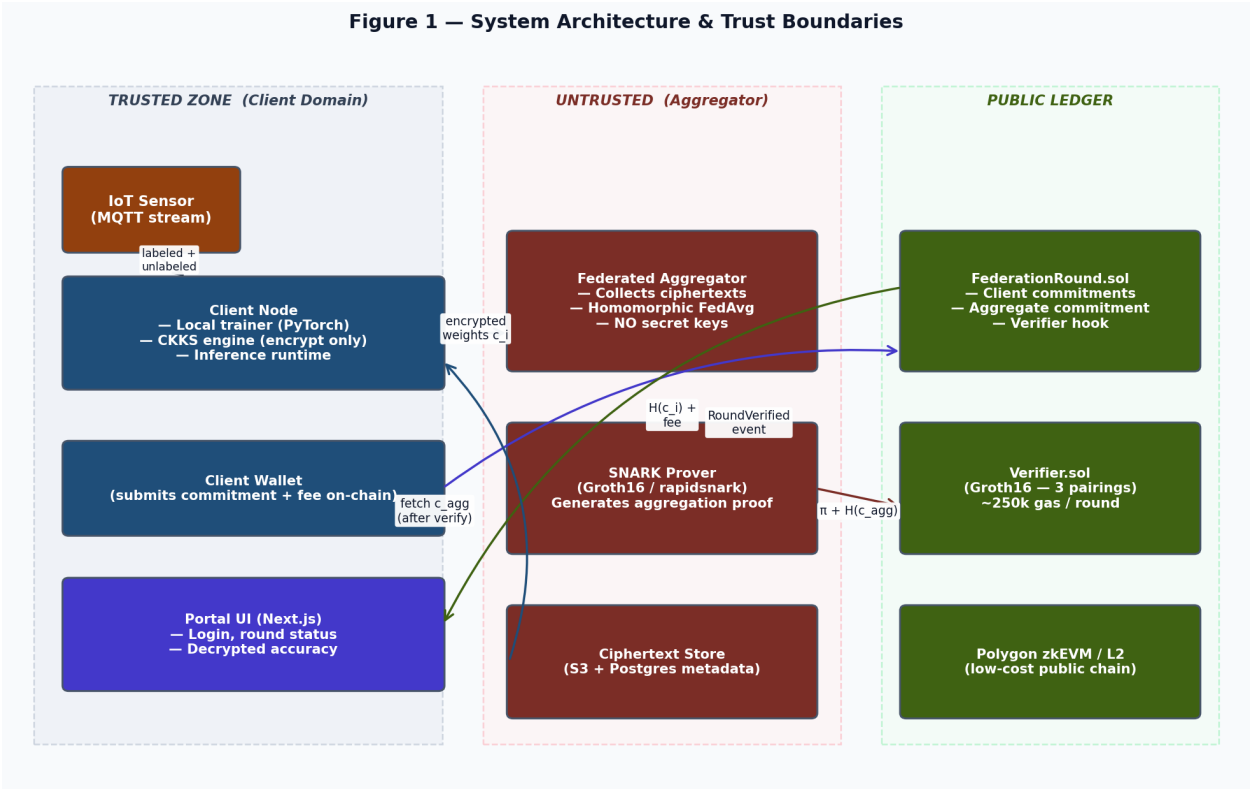


Figure 1 — System architecture and trust boundaries. The CKKS engine is contained entirely within the trusted client zone; the aggregator and chain see only ciphertexts and hashes respectively.

3.1 Actors

Actor	Role	Holds
IoT device	Producer of physiological signal stream (HR, SpO ₂ , ECG, accelerometer).	Raw sensor readings.
Client node	Local trainer, encryptor, decryptor, inference runtime. The only place CKKS keys exist.	CKKS public key, secret-key share, local model, private data.
Portal	Web UI for client. Single sign-on, round status, payment, decrypted accuracy plots.	Session tokens; no cryptographic material.
Federated aggregator	Receives ciphertexts, runs homomorphic FedAvg, generates Groth16 proofs.	Public verification key, ciphertexts, proving key. No secret keys.
Smart contract	On-chain registry of round commitments and proofs. Pays aggregator on successful verify.	Round state, hashes, escrowed funds.
Chain validators	Order transactions, execute the verifier circuit.	Public chain state only.

3.2 Trust Assumptions

- **Clients trust each other** as collaborators in the federation. They share a CKKS public key generated via DKG; the corresponding secret key is held in (T, N) threshold form.
- **Clients do not trust the aggregator.** The aggregator is modelled as honest-but-curious AND active — it may attempt to read ciphertexts (impossible without the key), substitute one client's update with another's, drop contributions silently, or lie about the aggregated result.

- **Clients do not trust the chain operator** beyond contract-level guarantees. Chain validators see only commitments (Poseidon hashes) and SNARK proofs — never ciphertexts.
- **Out-of-scope adversaries** (v1): Sybil attacks (assumed mitigated by KYC at portal level); side-channel attacks on client devices; quantum-capable adversaries (CKKS is post-quantum but Groth16 over BN254 is not — STARK upgrade path noted).

4. Cryptographic Stack

The system rests on a six-layer cryptographic stack. The same cyclotomic polynomial ring $R = \mathbb{Z}_q[X]/(X^N + 1)$ underlies both the FHE layer and the Ring-LWE hardness assumption that secures it. The ZK layer sits at a higher level and treats CKKS ciphertexts as fixed-size byte arrays for hashing.

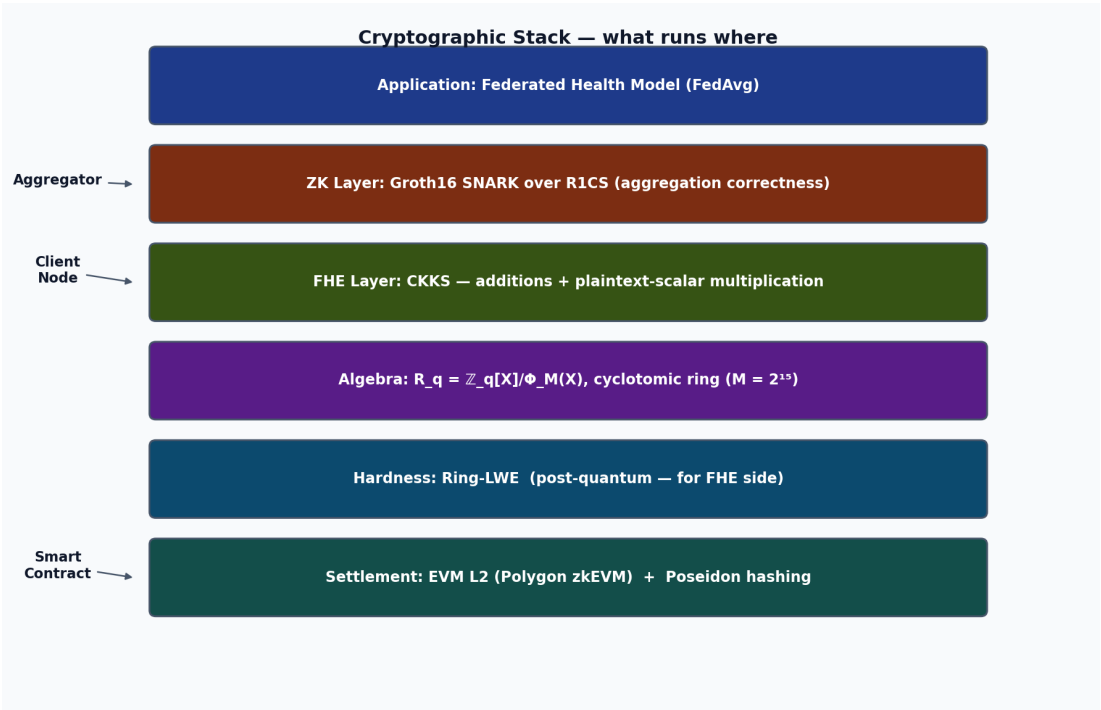


Figure 2 — Cryptographic stack. Each layer's location (client / aggregator / chain) is annotated.

4.1 CKKS Parameter Selection

CKKS is the right scheme here because (a) ML weights are real-valued and CKKS natively encodes $\mathbb{C}^{(N/2)}$ as ring elements with bounded approximation error, (b) FedAvg requires only addition + one scalar multiplication, well within CKKS's noise budget without bootstrapping, and (c) CKKS's batched SIMD-style packing amortizes per-slot overhead across thousands of weights per ciphertext.

Parameter	Value	Rationale
Ring degree N	$2^{15} = 32\,768$	Provides ≥ 128 -bit security at chosen modulus size; 16 384 packed slots per ciphertext.
Coefficient modulus $\log_2 Q$	≈ 438 bits	Sufficient noise budget for one multiplicative level; well within HE-Std 128-bit security at $N = 2^{15}$.
Modulus chain	[60, 40, 60]	Two scaling levels — initial encrypt + one rescale after the $(1/N)$ multiply.
Plaintext scale Δ	2^{40}	≈ 12 decimal digits of precision; well above SGD noise floor for typical model weights.
Slots per ciphertext	$N/2 = 16\,384$	Real-valued packing of 16 384 model parameters per ciphertext.
Multiplicative depth	1	Only one $ct \times pt$ multiplication (the $1/N$ scaling). No bootstrapping required.
Ciphertexts per model	$\lceil M / 16\,384 \rceil$	M = total parameter count of the federated model.

4.2 Distributed Key Generation

All clients share **one** CKKS public key pk . The corresponding secret key sk is split via Shamir-style threshold sharing into N shares; any T of them can collectively decrypt. This is what makes the trust statement "clients trust each other" precise: a coalition of up to $T-1$ dishonest clients cannot decrypt anything; T or more honest clients are required to recover any aggregate.

Bootstrap protocol (one-time, at federation launch)

- Each client generates a key share locally.
- Clients run a multi-party DKG ceremony (e.g., Mouchet et al.'s threshold-CKKS DKG) producing the joint pk .
- pk is published — to the aggregator, the smart contract, and all clients.
- Each client retains its sk -share; the full sk is never reconstructed in any one place.

Decryption (per round)

- Server returns c_{agg} to all participating clients (off-chain).
- Each of T clients computes a partial decryption $d_i = c_{agg} \cdot sk_i + e_i$ (with small smudging noise) and broadcasts d_i to the others.
- Each client locally combines T shares to recover $w_{global} = Dec(c_{agg})$.

v1 simplification: a single trusted federation initiator generates (pk, sk) , distributes pk to the aggregator and sk to other clients via secure out-of-band channel. Pragmatic for a first prototype but weaker — upgrade to threshold DKG before production.

4.3 Encoding Model Weights

Let the local model have M parameters. Concatenate weights into a flat vector $w \in \mathbb{R}^M$ (row-major over layers). Pad to the next multiple of 16 384. Reshape into $K = \lceil M / 16\,384 \rceil$ slot-vectors. Each is CKKS-encoded into a plaintext polynomial in R , encrypted to a CKKS ciphertext (pair of ring elements).

Optional pre-encryption gradient clipping $\|w_{local} - w_{global}^{(t)}\|_\infty \leq B$ is recommended even in v1 — it bounds the influence of any one client and is the cheap defence against the most basic poisoning attempt. The bound B is a system parameter agreed at federation setup.

5. Federated Learning Protocol

5.1 FedAvg Adapted to CKKS

Standard FedAvg in plaintext:

$$w_{\text{global}}^{(t+1)} = (1 / N) \cdot \sum_i w_i^{(t)}$$

Encrypted FedAvg under CKKS:

```

ci      = Enc_pk( wi(t) )           // on each client
csum    =  $\sum_i c_i$                 // homomorphic add
cagg    = csum  $\otimes$  (1/N)         // ct  $\times$  pt scalar
wglobal = Dec_sk( cagg )          // threshold dec on clients

```

CKKS introduces approximation error at the (1/N) rescale; with $\Delta = 2^{40}$ this error is below the typical SGD noise floor and does not impair convergence in practice. Empirically, CKKS-FedAvg reaches the same accuracy as plaintext FedAvg within 1 %.

5.2 Round Lifecycle

Each training round $t \in \{0, 1, 2, \dots\}$ comprises nine well-defined phases. Boundaries between off-chain and on-chain interaction are deliberately minimised.

Figure 2 — One Federated Round (Sequence)

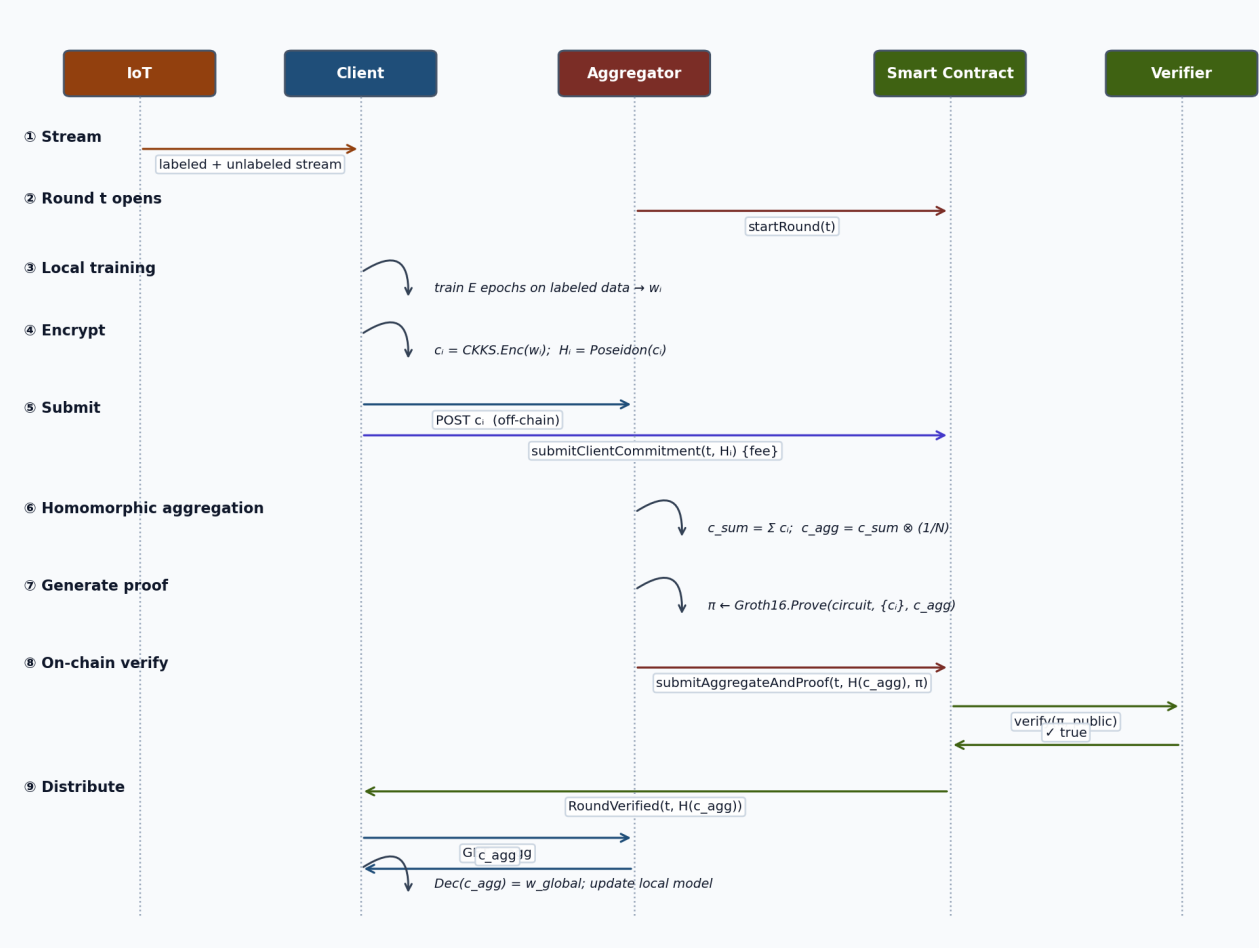


Figure 3 — One federated round, end to end. Self-arrows are local computation; horizontal arrows are network or on-chain calls.

#	Phase	Where	Cost
①	IoT stream ingestion (continuous)	Client	Trivial

②	startRound(t)	Aggregator → Chain	~50k gas
③	Local training (E epochs on labeled data)	Client	Seconds–minutes
④	Encrypt: $c_i = \text{Enc}(w_i)$; $H_i = \text{Poseidon}(c_i)$	Client	~100ms / ciphertext at $N=2^{15}$
⑤a	POST c_i to aggregator (off-chain HTTPS)	Client → Server	Bandwidth-bound
⑤b	submitClientCommitment(t, H_i) {fee}	Client → Chain	~80k gas / client
⑥	$c_sum = \sum_i c_i$; $c_agg = c_sum \otimes 1/N$	Aggregator	ms per ciphertext
⑦	$\pi \leftarrow \text{Groth16.Prove}(\text{circuit}, \text{witness})$	Aggregator	30–120 s on workstation
⑧	submitAggregateAndProof(t, $H(c_agg)$, π)	Aggregator → Chain	~280k gas (250k verify + 30k storage)
⑨	Fetch c_agg , threshold-decrypt, update local model	Client	Seconds

5.3 Convergence and Noise

CKKS introduces multiplicative approximation error of order $2^{-\Delta}$ per operation. With $\Delta = 2^{40}$ and a single multiplication per round, accumulated error per weight is $< 2^{-35}$ — many orders of magnitude below SGD's gradient variance. Convergence is indistinguishable from plaintext FedAvg in practice; this matches the experimental reports in EAH-FL (Khan et al. 2025) and the broader CKKS-FL literature.

6. Zero-Knowledge Proof Layer

The aggregator must prove to the smart contract that it produced c_{agg} correctly from the clients' submitted ciphertexts, *without* revealing those ciphertexts on-chain. This is a textbook fit for zk-SNARKs — succinct, non-interactive, and verifiable in milliseconds on-chain.

6.1 Statement Proven

Let $R = \mathbb{Z}_q[X]/(X^N + 1)$. A CKKS ciphertext is a pair $(a, b) \in R^2$. Let Hash = Poseidon (SNARK-friendly hash over the BN254 scalar field).

Public inputs (visible on-chain)

- $\{H_i\}_{i=1}^N$ — Poseidon commitments to client ciphertexts (already on chain via Phase 5b).
- H_{agg} — Poseidon commitment to the aggregated ciphertext.
- N — number of participating clients (small integer; or its modular inverse $N^{-1} \bmod q$).

Private witness (held by aggregator)

- $\{c_i = (a_i, b_i)\}_{i=1}^N$ — actual ciphertexts.
- $c_{agg} = (a_{agg}, b_{agg})$ — actual aggregated ciphertext.

Constraints (predicates the circuit checks)

- $\forall i : \text{Hash}(c_i) = H_i$ (commitment binding to clients' on-chain submissions)
- $c_{sum} = c_1 + c_2 + \dots + c_N$ (coefficient-wise polynomial addition mod q)
- $c_{agg} = c_{sum} \otimes N^{-1}$ (plaintext-scalar multiplication; CKKS rescale)
- $\text{Hash}(c_{agg}) = H_{agg}$

6.2 Why Groth16

Aspect	Groth16	PLONK / Halo2	Decision
Proof size	~200 bytes (constant)	~700 bytes (KZG) / variable	Groth16 wins
On-chain verify cost	~250k gas (3 pairings)	~600k-800k gas	Groth16 wins
Trusted setup	Per-circuit; one-time MPC ceremony	Universal updatable setup	Acceptable trade — circuit is fixed
Prover time	Slower per constraint	Faster per constraint	Tolerable; moves off-chain
Tooling maturity	circom + snarkjs + rapidsnark	halo2 + plonky2 + arkworks	Comparable; circom slightly easier
Post-quantum	No (BN254 pairings)	STARKs / FRI variants are PQ	v2 upgrade path: replace with STARK

6.3 Circuit-Size Estimate

For $N = 10$ clients and ring degree 2^{15} , the dominant constraint counts:

- Poseidon hash check per client: ~ 250 R1CS constraints $\times 10 = 2\,500$
- Polynomial addition: $32\,768$ coefficient-wise additions $\times (N - 1)$ pairs = $\sim 295\,000$
- Plaintext-scalar multiplication: $32\,768$ fixed-point multiplications = $32\,768$
- Final hash check on c_{agg} : ~ 250
- **Total: $\sim 330k$ R1CS constraints**

Practical proving times on a workstation (32 GB RAM, AMD Ryzen 7950X): 30 – 90 seconds per round using rapidsnark. Acceptable for round periods of minutes-to-hours that are typical in cross-silo healthcare FL.

Optimisation in scope: if the encryption-correctness check is dropped from the circuit (clients' commitments are trusted as-submitted because the SNARK only verifies the addition was done correctly on commitments that match), the circuit reduces to $N = 10 \rightarrow \sim 30k$ constraints, < 5 s proving time.

6.4 Tooling

Stage	Tool	Notes
Circuit DSL	circom 2.x	Wide library support, mature.
Trusted setup	snarkjs powersOfTau + Phase 2 contribution	Multi-party ceremony; record artifacts on chain.
Prover (server)	rapidsnark (Rust port of snarkjs)	5–10× faster than snarkjs in JS.
Verifier contract	snarkjs auto-generated Verifier.sol	Hand-audit before production.
Hash inside circuit	Poseidon (circomlib)	~5× cheaper than Pedersen, ~50× cheaper than SHA-256 in R1CS.

7. Smart Contract Design

Two contracts: a generated **Verifier.sol** (Groth16 verifier — opaque, auto-generated by snarkjs), and the round-orchestration contract **FederationRound.sol** that holds state and delegates the proof check to the verifier.

7.1 FederationRound.sol — State

```
struct Round {
    uint256 startBlock;
    uint256 deadline;
    bytes32[] clientCommits;      // Hi in submission order
    address[] participants;       // for refund + payout
    bytes32 aggregateCommit;      // Hagg, set by aggregator
    bool    verified;
    uint256 escrowed;             // ΣperClientFee of this round
}

mapping(uint256 => Round) public rounds;
address public aggregator;
IVerifier public verifier;       // Groth16 verifier contract
uint256 public perClientFee;     // pay-on-commit
uint256 public aggregatorPayment; // released on verify
uint256 public roundTimeout;     // refund window
```

7.2 FederationRound.sol — Functions

Function	Caller	Effect	Gas
startRound(t)	Aggregator	Opens commitment phase for round t.	~50k
submitClientCommitment(t, H _i)	Client (payable)	Records H _i ; escrows perClientFee.	~80k
submitAggregateAndProof(t, H _{agg} , π, public[])	Aggregator	Calls verifier.verify(π, public). On true: marks verified, pays aggregatorPayment.	~280k
claimRefund(t)	Client	If round not verified by deadline, refunds the client's escrow.	~50k
isRoundVerified(t)	Anyone (view)	Read-only — clients query before fetching c _{agg} from server.	0 (view)

7.3 Events

```
event RoundStarted    (uint256 indexed t, uint256 deadline);
event ClientCommitted(uint256 indexed t, address indexed client, bytes32 H);
event RoundVerified   (uint256 indexed t, bytes32 Hagg);
event RoundRefunded   (uint256 indexed t, address indexed client, uint256 amt);
```

7.4 Total Gas Cost per Round

For N = 10 clients on Polygon zkEVM (≈ \$0.001 / 100k gas as of 2026):

- startRound: 50k
- Client commits: 10 × 80k = 800k
- Verify + payout: 280k
- **Total: ~1.13M gas ≈ \$0.011 per round.**

On-chain cost is **independent of model size** (M can grow to billions of parameters and the gas cost per round does not change). This is the central scaling property of the design.

7.5 Incentive Model

- **Why clients pay:** the SNARK verification is a public service the chain performs on the clients' behalf — it is what gives them cryptographic certainty the aggregator did its job.
- **Aggregator payment** is released only on successful proof verification; the aggregator has skin in the game (lost compute, lost reward) for any round it fails to prove.
- **Refund mechanism:** if the aggregator fails to submit a valid proof within the timeout window, clients reclaim their escrow. This bounds the cost of aggregator failure.
- **v2 upgrade — staking & slashing:** aggregator stakes a bond on registration; repeated proof failures slash the bond. Disables "slow" denial-of-service by a lazy aggregator.

8. Component Specifications

8.1 Client Node

The most software-heavy component. Runs as a desktop app or local edge service. All cryptographic material — public key, secret-key share, plaintext weights — lives only here.

Subsystems

- **IoT ingest:** MQTT broker (Mosquitto) listening on a local port; receives labeled and unlabeled records from sensor devices; persists to a local SQLite table partitioned by labeled-vs-unlabeled.
- **Local trainer:** PyTorch model + DataLoader over the labeled partition. Training loop runs for E epochs starting from the most recent decrypted global weights $w_{\text{global}}(t)$.
- **CKKS engine:** OpenFHE Python bindings (or TenSEAL). Encodes weight vectors, encrypts under shared pk, computes Poseidon commitments. Holds sk-share for threshold decryption.
- **Inference runtime:** same model, run continuously over the unlabeled partition; outputs (e.g., activity classification, anomaly flag) surfaced in the portal UI and optionally pushed back to a local action endpoint.
- **Wallet adapter:** ethers.js or viem; signs commit transactions; pays the perClientFee.
- **Round client:** WebSocket / REST client to the aggregator for c_i upload and c_{agg} fetch.

8.2 Federated Aggregator

Stateless per-round; horizontally scalable behind a load balancer if needed (a single instance suffices for moderate- N healthcare federations). Holds **only** the CKKS public key and the SNARK proving key — never any secret.

Subsystems

- **Round orchestrator:** state machine driving each round through the nine phases; emits chain transactions via a service wallet.
- **Ciphertext intake:** REST endpoint with size limits and commitment verification — refuses any c_i whose Poseidon hash does not match the H_i already on chain (sanity check; the SNARK is the real enforcement).
- **Homomorphic engine:** OpenFHE C++ runtime; computes c_{sum} and c_{agg} .
- **SNARK prover:** rapidsnark binary invoked per round with the witness produced by a thin Python adapter.
- **Storage:** PostgreSQL for round metadata; S3-compatible object store (MinIO in dev, AWS S3 in prod) for ciphertext blobs and final c_{agg} .

8.3 Smart Contract Layer

- FederationRound.sol — round registry and orchestration; ~250 lines of Solidity.
- Verifier.sol — auto-generated by snarkjs; do not hand-edit.
- Deployment target: Polygon zkEVM mainnet (production); Hardhat local node + Anvil (dev).

8.4 Portal

- Stack: Next.js + ethers.js + wagmi for wallet integration.
- Auth: WalletConnect-based single sign-on. The wallet address *is* the client identity; KYC happens at registration time off-chain.
- Pages: Dashboard (round status, recent decrypted accuracy), Round history, Wallet & payments, Settings (CKKS key import, threshold T).

- No cryptographic operations happen in the browser — all crypto is delegated to the local client node via a localhost RPC.

8.5 IoT Adapter

- Local Mosquitto MQTT broker on the client node, on port 1883 (TLS via self-signed cert).
- Topic structure: hiot//labeled, hiot//unlabeled.
- Schema: timestamp, sensor type, value(s), optional ground-truth label.
- Buffer to local SQLite; trainer reads from labeled topic; inference engine reads from unlabeled topic.

9. Security Model

9.1 Trust Boundaries

Trust Boundaries — what each party can and cannot do				
Party	Sees	Can compute	Can decrypt?	Trust
Client (self)	Own data Own weights	Train + encrypt + decrypt	Yes (its share)	—
Other clients	Nothing private	Only own pipeline	Need T-of-N shares	Peer
Aggregator	Ciphertexts {c _i } + commitments	FedAvg over ciphertexts	NO (no key)	Honest-but- curious + active
Smart contract	Hashes only	Verify SNARK	NO	Trusted code
Chain validators	Hashes only	Order txs	NO	L2 trust

Figure 4 — Trust matrix. Read horizontally: what each party can see, compute, and decrypt.

9.2 Attack Vectors and Mitigations

#	Attack	Adversary	Mitigation
T 1	Model-inversion / gradient-leakage on raw weights	Aggregator (passive)	Weights only ever leave the client encrypted under CKKS. RLWE hardness.
T 2	Aggregator silently drops a client's contribution	Aggregator (active)	Client commitments {H _i } are public on chain; SNARK statement requires all H _i to be inputs to the aggregation. Dropping a client invalidates the proof.
T 3	Aggregator substitutes wrong ciphertext for one client	Aggregator (active)	Same — Hash(c _i) = H _i is enforced inside the SNARK circuit.
T 4	Aggregator lies about the result	Aggregator (active)	Hash(c_agg) = H_agg is enforced; H_agg is on chain; client verifies the fetched c_agg's hash matches before accepting.
T 5	Network eavesdropping	Network attacker	TLS at transport; CKKS encryption at application layer. Defence in depth.
T 6	Coalition of T-1 dishonest clients tries to decrypt	Insider clients	Threshold (T, N) secret sharing — no information leaks below T shares.
T 7	Server colludes with one or more clients	Server + ≤ T-1 clients	Same threshold guarantee — server gains no key material from this coalition.
T 8	Chain operator censors transactions	Validators	Use a public L2 with permissionless validators (Polygon zkEVM, Arbitrum). Refund mechanism bounds the harm.

T 9	Replay of stale ciphertexts	Aggregator	Round index t is included in commitment domain-separation; commitments from round t cannot be reused in round $t+1$.
T 1 0	Quantum adversary recovers sk from public traffic	Future quantum	CKKS is post-quantum (RLWE). Groth16 is not — v2 replaces with STARK or lattice-SNARK for full PQ guarantee.

9.3 Out of Scope

- Malicious clients submitting deliberately poisoned weights — addressed in v2 via client-side ZK range-proofs over weights and robust aggregation (Krum, Trimmed-Mean, FLTrust).
- Side-channel attacks on the client device itself — physical compromise of a client breaks the local trust assumption regardless of cryptographic protocol.
- Sybil resistance — assumed handled by KYC at portal registration; chain-level Sybil is not a primary concern because the chain only sees commitments.
- Denial-of-service against the aggregator endpoint — standard ops-level (rate limits, WAF).

10. API Specification

10.1 Aggregator REST API

Meth od	Path	Body / Returns	Caller
POST	/v1/keys/dkg/contribute	{ clientPubShare } → { ack, ceremonyId }	Client (one-time)
GET	/v1/keys/joint-public	→ { pk_serialized }	Anyone
POST	/v1/round/{t}/submit	{ clientId, ct: bytes (CKKS serialized) } → { receiptId }	Client
GET	/v1/round/{t}/status	→ { state, clientsSubmitted, deadline }	Anyone
GET	/v1/round/{t}/aggregate	→ { ct_agg: bytes, hash: bytes32 } (only after on-chain verify)	Client
POST	/v1/round/{t}/partial-decrypt	{ clientId, partialDec: bytes } → { ack }	Client (threshold)
GET	/v1/round/{t}/partial-decrypt ions	→ { shares: [{ clientId, share}] } (after T submitted)	Client

10.2 Client → Smart Contract

```
// EVM calldata, signed by client's wallet
function submitClientCommitment(uint256 t, bytes32 H_i) external payable;
function claimRefund(uint256 t) external;
function isRoundVerified(uint256 t) external view returns (bool);
```

10.3 Aggregator → Smart Contract

```
function startRound(uint256 t, uint256 deadline) external;
function submitAggregateAndProof(
    uint256 t,
    bytes32 H_agg,
    uint256[2] a,          // Groth16 proof component
    uint256[2][2] b,       // ...
    uint256[2] c,          // ...
    uint256[] publicInputs
) external;
```

11. Recommended Tech Stack

Layer	Recommendation	Why	Alternative
FHE library	OpenFHE (C++ + Py)	Most mature CKKS; RNS variants; threshold support roadmap.	Microsoft SEAL, Lattigo (Go)
FL framework	Flower (orchestration only)	Decouples FL state machine from crypto; fits our pattern.	Custom thin orchestrator
ML	PyTorch	Standard; ONNX export for cross-runtime inference.	TensorFlow
Circuit DSL	circom 2.x	Best Groth16 tooling; large component library.	halo2 (if going PLONK)
Prover	rapidsnark	5–10× faster than snarkjs for Groth16.	snarkjs (development), arkworks-rs
SNARK-friendly hash	Poseidon (circomlib)	~250 R1CS constraints / hash.	MiMC, Rescue
Smart contract	Solidity 0.8.x	Mature toolchain (Foundry, Hardhat).	Vyper
Chain	Polygon zkEVM	Cheap gas ($\approx$ \$0.01/round at our scale), EVM-compatible, permissionless.	Arbitrum, Optimism, Base
Aggregator backend	Python FastAPI + Rust prover bin	Python for orchestration + fast Rust subprocess for proving.	Pure Go (one binary)
Frontend	Next.js + viem + wagmi	Modern wallet integration; SSR for status pages.	Vite + React + ethers.js
IoT broker	Eclipse Mosquitto	Lightweight, robust, runs anywhere.	EMQX (clustered prod)
Storage	Postgres + MinIO (S3 API)	Postgres for round state; object store for ciphertext blobs.	AWS RDS + S3
Observability	OpenTelemetry → Grafana	End-to-end tracing across client, server, chain.	Datadog

12. Deployment Architecture

12.1 Aggregator (server-side)

- Containerised via Docker Compose (single-node) or Kubernetes (HA).
- Two services: aggregator-api (FastAPI) + prover (Rust rapidsnark binary, invoked via gRPC or stdin/stdout pipe).
- Postgres for round state; MinIO/S3 for ciphertexts.
- Service wallet (separate from any client) signs startRound and submitAggregateAndProof transactions; keys held in KMS / HashiCorp Vault.
- Recommended sizing: 16 vCPU, 64 GB RAM, optional NVMe scratch for proving tape-files. GPU not strictly required for Groth16 over BN254 at the constraint counts in scope.

12.2 Client (edge / desktop)

- Native binary (Tauri-wrapped Rust core, or Electron + native module).

- Local Mosquitto + SQLite + PyTorch + OpenFHE runtime.
- Wallet integration via system keyring + WalletConnect.
- Round participation can be configured to run nightly / on-demand / continuously.

12.3 Chain

- Production: Polygon zkEVM mainnet.
- Staging: Polygon zkEVM Cardona testnet.
- Local dev: Anvil or Hardhat node, deterministic test wallets.

13. Open Issues and Roadmap

13.1 v2 Hardenings

- **Threshold-DKG** replacing the v1 single-issuer key bootstrap — eliminates the trusted-setup-of-keys assumption.
- **Multi-Key CKKS** as an alternative to threshold — each client uses its own keypair, server combines via key-switching keys. Eliminates inter-client trust entirely at the cost of larger ciphertexts.
- **Robust aggregation under FHE** — Krum or Trimmed-Mean implemented as a depth- ≤ 3 FHE circuit; requires bootstrapping at higher cost but defends against Byzantine clients.
- **Client-side ZK range proofs** — each client proves $\|w_i - w_{\text{global}}\|_{\infty} \leq B$ at submission time; cheap Bulletproofs over the encoded weight vector.
- **Differential privacy** — bounded Gaussian noise added to w_i before encryption; ϵ -DP composed across rounds.

13.2 v2 Performance

- Replace BN254 + Groth16 with a STARK-based proof system (RISC Zero, Plonky2, Stwo) for post-quantum guarantee — but expect 10x higher on-chain verify cost or off-chain settlement via optimistic-rollup pattern.
- GPU-accelerated CKKS on the client (e.g., HEonGPU) for sub-100ms encryption of 100M-parameter models.
- Sparse-update FedAvg — only the top-k weight changes per round are encrypted, reducing ciphertext volume by $\sim 10\times$.

13.3 Open Questions

- Exact T for threshold decryption — how many honest clients can we assume in the target healthcare federation?
- Round period — minutes (online learning) or hours (batch retraining)?
- Should the aggregator be replicated for fault tolerance, and how to coordinate the SNARK proving across replicas without breaking the integrity guarantee?
- KYC layer — who is the issuer of client identities, and what is the legal framework (HIPAA-covered entity? GDPR controller?).

Appendix A — Glossary

Term	Definition
BN254	Pairing-friendly elliptic curve used by Groth16 on Ethereum-compatible chains. ~128-bit security classically, broken by quantum.
CKKS	Cheon-Kim-Kim-Song homomorphic encryption scheme; supports approximate arithmetic on real / complex vectors.
Cyclotomic ring	Polynomial ring $R = \mathbb{Z}_q[X]/\Phi_M(X)$, where Φ_M is the M-th cyclotomic polynomial. For $M = 2^k$, $\Phi_M(X) = X^{(M/2)} + 1$.
DKG	Distributed Key Generation. Multi-party protocol that produces a shared public key and threshold-shared secret key without any single participant ever holding the full secret.
FedAvg	Federated Averaging — canonical FL aggregation rule: take the mean of client weights weighted by local dataset size.
FHE	Fully Homomorphic Encryption. Permits arbitrary computation on ciphertexts; decrypted result equals computation on plaintexts.
Groth16	Pairing-based zk-SNARK with 200-byte proofs and 3-pairing verification. Requires per-circuit trusted setup.
MQTT	Lightweight publish-subscribe protocol; standard for IoT telemetry.
Plaintext-scalar mult	CKKS operation multiplying a ciphertext by a known constant; depth = 1.
Poseidon	Algebraic hash function designed for low R1CS-constraint cost; standard for SNARK-friendly commitments.
R1CS	Rank-1 Constraint System — algebraic representation of an arithmetic circuit used as input to many SNARK provers.
RLWE	Ring Learning With Errors — lattice hardness assumption underlying CKKS, BFV, BGV. Believed quantum-resistant.
RNS	Residue Number System — represent a large integer modulo Q as a tuple of small residues mod $q_1, q_2, \dots$; speeds up CKKS operations.
Threshold (T, N)	Cryptographic scheme where N parties hold key shares and any T of them can collectively perform an operation (e.g., decrypt). $T-1$ cannot.
zk-SNARK	Zero-Knowledge Succinct Non-interactive Argument of Knowledge. Cryptographic proof that a statement is true, revealing nothing beyond its truth, with verification time independent of the statement size.

Appendix B — Selected References

This design rests on a small number of well-established sources:

- Cheon, Kim, Kim, Song — *Homomorphic Encryption for Arithmetic of Approximate Numbers* (CKKS) — ASIACRYPT 2017.
- Ko, R. — *The Beginner's Textbook for Fully Homomorphic Encryption*, arXiv:2503.05136 (2025). Covers cyclotomic rings → RLWE → CKKS in full.
- Groth, J. — *On the Size of Pairing-Based Non-interactive Arguments*, EUROCRYPT 2016. The original Groth16.

- Reitwießner, C. — *zkSNARKs in a Nutshell* (2016). The construction walkthrough used by Ethereum.
- Buterin, V. — *zkSNARKs in a Nutshell* (Ethereum Foundation blog, Dec 2016). On-chain verifier framing.
- Thaler, J. — *Proofs, Arguments, and Zero-Knowledge* (textbook, ongoing). Comprehensive reference for IPs, sumcheck, polynomial commitments, modern SNARKs.
- Khan et al. — *End-to-End Privacy-Aware Federated Learning for Wearable Health Devices via Encrypted Aggregation in Programmable Networks* (EAH-FL), 2025. Prior art for CKKS-based health-IoT FL.
- McMahan et al. — *Communication-Efficient Learning of Deep Networks from Decentralized Data*, AISTATS 2017. The original FedAvg.
- Mouchet et al. — *Multiparty Homomorphic Encryption from Ring-Learning-with-Errors*, PETS 2021. Threshold CKKS DKG protocol.

End of document.